

FPGA TP : Digital clock in VHDL

Groups of 1 or 2 (preferred) ,

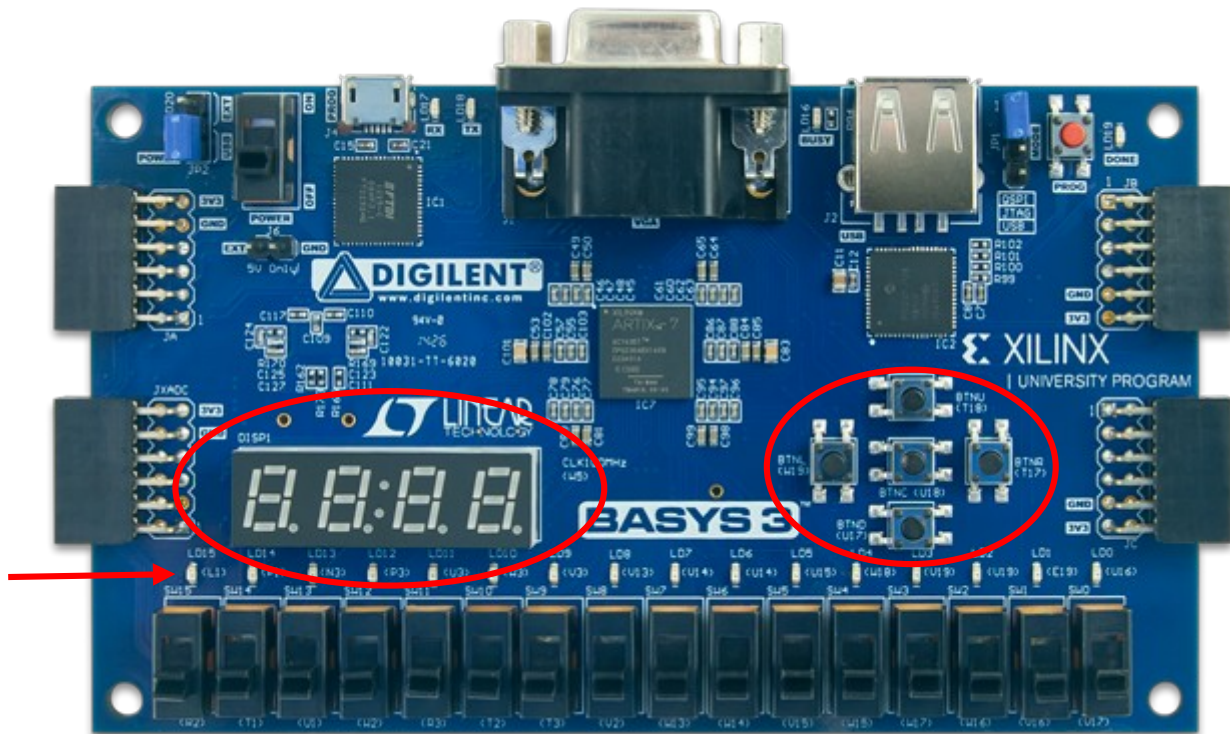
Duration : 3 weeks, report to submit by end of week 7

Objectives :

- design in VHDL
- simulate (whichever tool you prefer)
- synthesize to Basys 3 board (use Vivado)

NB : the Basys 3 has the following I/O abilities :

- 4 “7-segments” digits
- 4 push buttons
- (eventually additional leds)



NB : 5th (central button) is normally used for reset
Switches will not be used.

The clock should have the following functions

- Normal time counting (HH:MM:SS(.T)) – main input clock will be period of 0.1 sec
this choice of clock period is driven by
 - time reaction to user interaction (button pushing)
 - precision of the clock
 - digits blinking during adjustment modes
- !! use BCD counting : one counter for each digit

- Time setting and adjustment

!! during time setting, the clock should continue working (for example in case of summer time adjustment, you keep the exact time precision while simply changing hour)
three adjustment modes : increment hour, increment minute, reset second+tenth (for precise adjustment).

- (optional if you have time, not mandatory) : Alarm

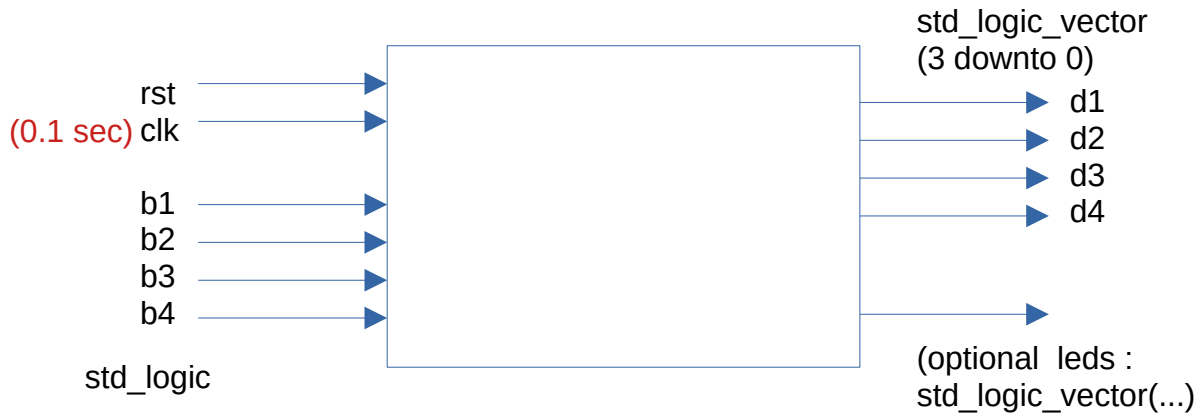
Alarm should be activated or not, hour and minute should be set, so three additional adjustment modes : alarm on/off, increment alarm hour, increment alarm minute

When current time (H:M) equals alarm set time (and alarm is active) then alarm start buzzing .
Buzzer is switch off by pushing a button

NB there is no sound buzzer on the Basys 3 , so you can either use an additional led, or a special code displayed on the main 4 digits display.

Of course, normal time counting should continue during alarm setting !

The main circuit interface (VHDL ENTITY) should be as followed :



Each of the 7-segments digits should be able to display : any decimal number (0 to 9), a blank character (for blinking), or eventually special characters (like A and L for “alarm” or n and F for “on” and “off”)

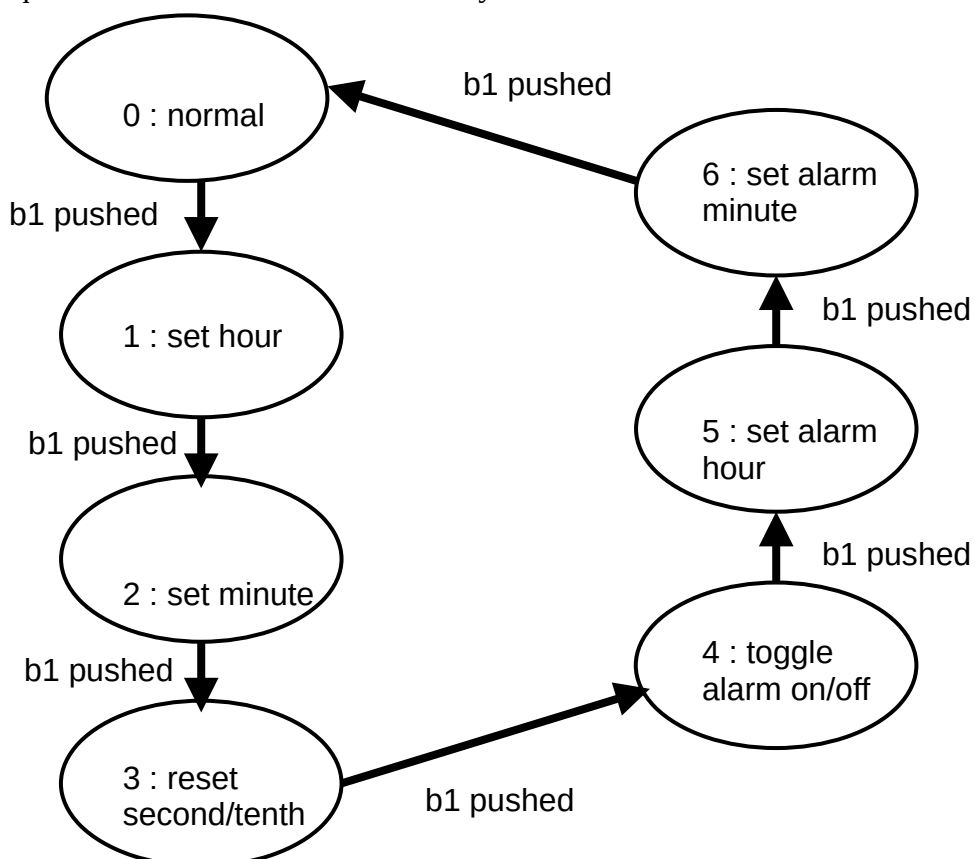
=> 4 bits vectors give 16 possibilities, should be enough

Buttons must be synchronised to the 0.1 sec clock.

If pushed for more than 0.1 sec, it should be considered as a single push until button is released and again pressed.

You can optionally code long push for fast incrementation, but it is not mandatory.

The main sequence state machine is controlled by button b1 :



The effect of b2 depends on the main mode :

0 : normal : change display from HHMM to SS ; if alarm buzzer is running, stop it

1 : set hour => increment hour

2 : set minute => increment minute

3 : reset second/tenth : to 0 (for precision adjustment)

4 : toggle alarm mode

5 : set alarm hour => increment alarm hour

6 : set alarm minute => increment alarm minute

An example for the digits displayed (NB : indications in red means blinking, for example 0.7 sec on, 0.3 sec off) – you are free to propose other solutions, eventually using additional leds

0 : HHMM if b2 not pushed, --SS if b2 pushed , (for example HHMM if buzzer running)

1 : HHMM

2 : HHMM

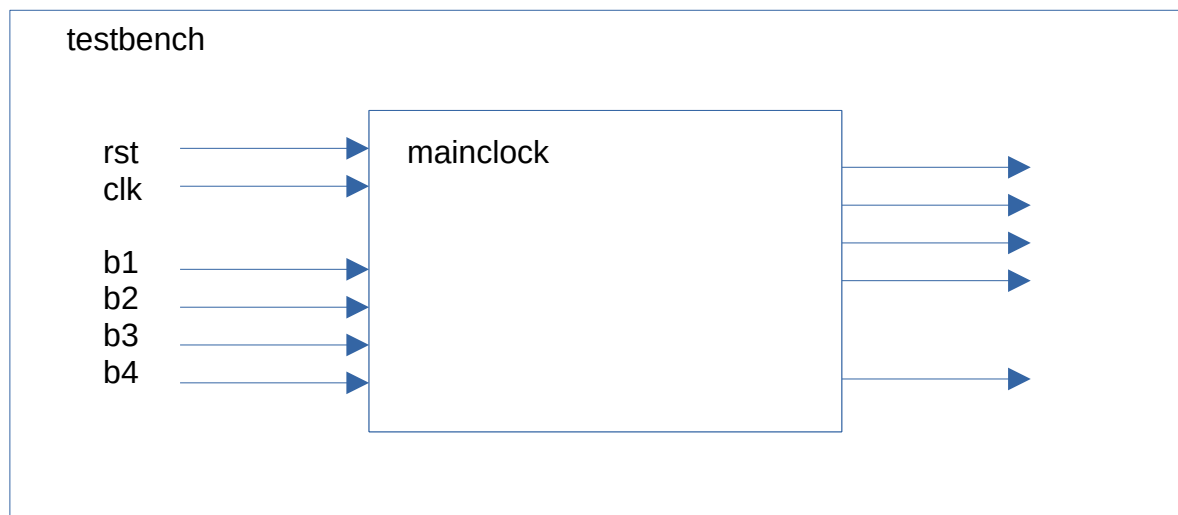
3 : --SS

4 : AL0n or AL0F

5 : HHAL (for example)

6 : ALMM (for example)

For simulation, you can of course encapsulate the main clock circuit in a testbench



or use interactive simulation commands like in Vivado

For synthesis additional logic blocks are required :

- a 7-segment sequencer and decoder to access the 4-digits display

- two clock dividers : one for the 0.1 sec main clock, the other for driving the 7-segment sequencer (about 1 kHz)

If you have no time for doing this, I will provide you with the top-level synthesis source.

